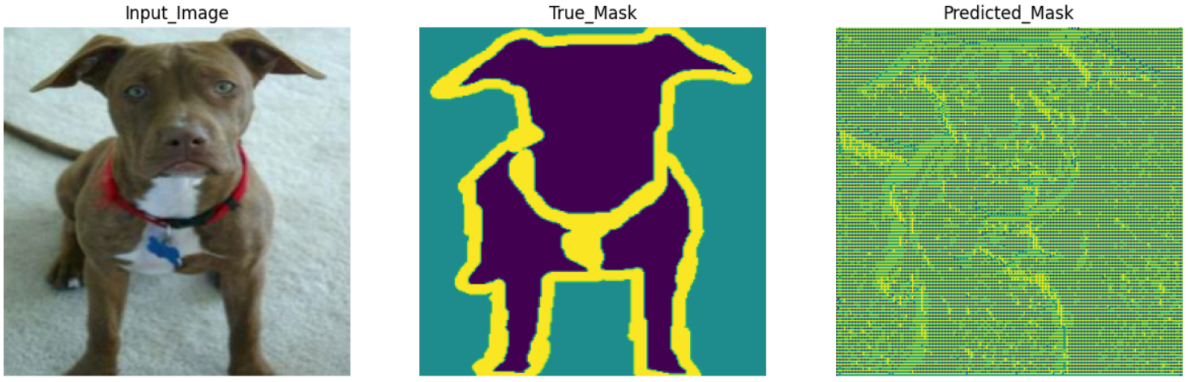
Lab 6: Image Segmentation

In this lab, we will learn how to implement and train an Autoencoder based model (A Simple UNET architecture) to segment pet images on “oxford_iiit_pet” dataset. A simple baseline model has been provided to you on your tutorial notebook. Here we would break down the code structure for you for better understanding.

Below is an example of a dog image and the true segmentation mask that we are interested to predict and the predicted mask on the extreme right show predictions without any training. Our objective is to make the predicted mask as close to true mask as possible.



Let us first inspect the dataset after downloading it online, let us see what the dataset contains and what data exactly do we need for training this model, since we have loaded the dataset using tensorflow dataset, we can check the structure of the dataset by printing ds_info:

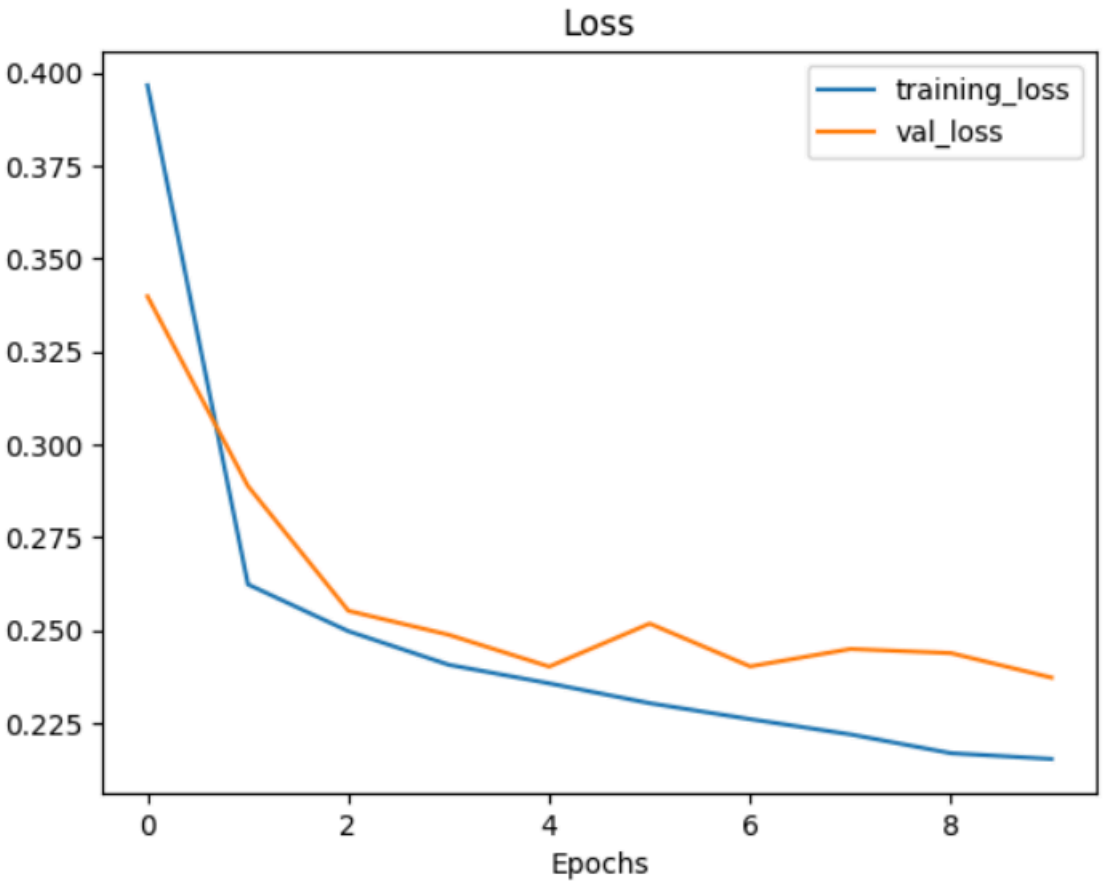
```
#download the dataset
(train_data, test_data), ds_info = tfds.load('oxford_iiit_pet:3.*.*',
                                             with_info=True,
                                             split=['train', 'test'])
```

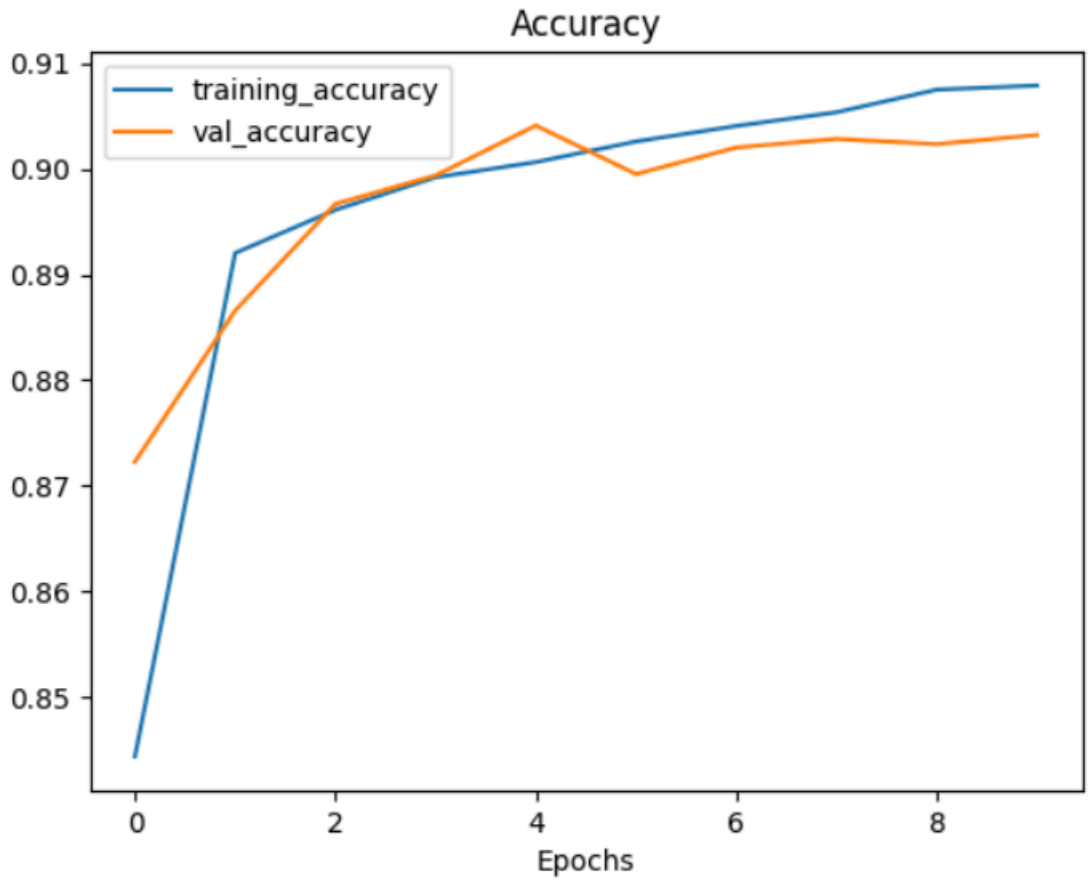
```
#information
ds_info
```

```
tfds.core.DatasetInfo(
    name='oxford_iiit_pet',
    full_name='oxford_iiit_pet/3.2.0',
    description="""
    The Oxford-IIIT pet dataset is a 37 category pet image dataset with roughly 200
    images for each class. The images have large variations in scale, pose and
    lighting. All images have an associated ground truth annotation of breed.
    """,
    homepage='http://www.robots.ox.ac.uk/~vgg/data/pets/',
    data_dir=PosixGPath('/tmp/tmpklxdso35tfds'),
    file_format=tfrecord,
    download_size=773.52 MiB,
    dataset_size=774.69 MiB,
    features=FeaturesDict({
        'file_name': Text(shape=(), dtype=string),
        'image': Image(shape=(None, None, 3), dtype=uint8),
        'label': ClassLabel(shape=(), dtype=int64, num_classes=37),
        'segmentation_mask': Image(shape=(None, None, 1), dtype=uint8),
        'species': ClassLabel(shape=(), dtype=int64, num_classes=2),
    }),
    supervised_keys=('image', 'label'),
    disable_shuffling=False,
    splits={
        'test': <SplitInfo num_examples=3669, num_shards=4>,
        'train': <SplitInfo num_examples=3680, num_shards=4>,
    },
    citation="""@InProceedings{parkhi12a,
      author       = "Parkhi, O. M. and Vedaldi, A. and Zisserman, A. and Jawahar, C.~V.",
      title        = "Cats and Dogs",
      booktitle    = "IEEE Conference on Computer Vision and Pattern Recognition",
      year         = "2012",
    }""",
)
```

We see the only thing important for us here is the feature dictionary which is nothing but a python dictionary with keys and values. Since we're only interested in the original images and the segmentation masks we would only retrieve these two keys highlighted above. Please note, if you plan to use any other dataset the keys would be different.

We build the model with just a few layers to build a small Unet model and train it for 10 epochs for demonstration. Once trained we plot the loss and accuracy of the model using helper functions,


Loss
training_loss
val_loss
0.400
0.375
0.350
0.325
0.300
0.275
0.250
0.225
0
2
4
6
8
Epochs


Accuracy
training_accuracy
val_accuracy
0.91
0.90
0.89
0.88
0.87
0.86
0.85
0
2
4
6
8
Epochs

We show the prediction of the trained model against the true mask this time and here you can see a significant difference in predicted mask,

```
[ ] show_predictions(test_batches, 3)
```

```
1/1 [==============================] - 0s 49ms/step
```


```
1/1 [==============================] - 0s 48ms/step
```


```
1/1 [==============================] - 0s 48ms/step
```

Finally, Any random pet image from the internet is downloaded and we test our model on that, let's see how good our model prediction is on a real life image outside our dataset, this is also demonstrated in the tutorial notebook as well,

```
1/1 [==============================] - 0s 33ms/step
```

Objective:

Given all these examples to you, your job in this lab is to ***improve*** *the* ***performance*** *of the model compared to the baseline model.*

Options to explore:

- Train for more epochs

- Modify learning rate (LR)

- Modify the Optimizer (SGD, Adam, ...)

- Learning Rate Schedulers (Cosine Annealing)

Also, You're encouraged to change the dataset from oxform iit pet to any other dataset online available on the internet or on tfds. You can further compare the loss and accuracy of this baseline model vs your improved models based on the changes you have made.

You can further analyse other performance matrices other than accuracy such as, precision, recall, F1 score.

*You need to submit a comprehensive report along with results comparing this baseline mode*l and your models highlighting your methodology. If model 1 (based on some changes) does not improve the model performance you can still mention that in your report but justify your answer why it did not work. You can check the list of datasets available on tfds using this code,

```
[ ]  tfds.list_builders()
```

Check this url to find more information about the datasets,

https://www.tensorflow.org/datasets/catalog/overview